

Lemma unique_max (A : R -> Prop) (x y : R) :
x is_a_max_of A -> y is_a_max_of A -> x = y.

Proof.

intros [HxA Hx].

intros [HyA Hy].

apply Rle_antisym.

exact (Hy x HxA).

exact (Hx y HyA).

Qed.

1. Term
Extraction

2. Gemini 2.5
Annotation

```
fun (A : R -> Prop) (x y : R) (H : x is_a_max_of A) =>  
  match H with  
  | conj x0 x1 =>  
    (fun (HxA : A x) (Hx : up_bounds A x) (H0 : y is_a_max_of A)  
    =>  
      match H0 with  
      | conj x2 x3 => (fun (HyA : A y) (Hy : up_bounds A y) =>  
        Rle_antisym x y (Hy x HxA) (Hx y HyA))  
        x2 x3 end) x0 x1 end : forall (A : R -> Prop) (x y : R),  
        x is_a_max_of A -> y is_a_max_of A -> x = y
```

Okay, let's try to transform this proof term into a sequence of coq tactics. First let's write down the hypotheses, and the initial goal (after the "|" symbol) given by the coq proof assistant: [...]

The proof term starts with [...]. The lambda term names them `HxA` and `Hx`. So, a good first step would be `intros [HxA Hx]`. This will move `x is\_a\_max\_of A` into the context, and I'll have two new hypotheses, `HxA` and `Hx`. The goal will then become `y is\_a\_max\_of A -> x = y`.*)

intros [HxA Hx].

Alright, after the first step, my proof state should look like this:

[...] So, I'll use the `intros` tactic again: `intros [HyA Hy]`. This will give me two new hypotheses, `HyA : A y` and `Hy : forall z : R, A z -> z <= R y`. The goal will be updated to `x = y`. [...]